

```

#include <windows.h>
#include <d3d11.h>
#include <d3dcompiler.h>

#pragma comment(lib, "d3d11.lib")
#pragma comment(lib, "d3dcompiler.lib")

// Структура вершины для нашего видеоэкрана
struct SimpleVertex
{
    float Pos[3]; // Координаты X, Y, Z в пространстве экрана
    float Tex[2]; // Текстурные координаты U, V для видеопотока
};

// --- ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ (СТРОГО ПО ОДНОМУ ЭКЗЕМПЛЯРУ): ---
ID3D11Device* g_pd3dDevice = NULL; // Видеокарта
ID3D11DeviceContext* g_pImmediateContext = NULL; // Контекст команд
IDXGISwapChain* g_pSwapChain = NULL; // Буфер экрана
ID3D11RenderTargetView* g_pRenderTargetView = NULL; // Окно вывода

ID3D11VertexShader* g_pVertexShader = NULL; // Вершинный шейдер
ID3D11PixelShader* g_pPixelShader = NULL; // Пиксельный шейдер

ID3D11InputLayout* g_pVertexLayout = NULL; // Формат вершин
ID3D11Buffer* g_pVertexBuffer = NULL; // Буфер геометрии

ID3D11ShaderResourceView* g_pTextureSRV = NULL; // Наша текстура 2x2
ID3D11SamplerState* g_pSamplerState = NULL; // Жесткий сэмплер (Point)

LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam);

// Функция для компиляции HLSL-файлов
HRESULT CompileShaderFromFile(const WCHAR* szFileName, LPCSTR szEntryPoint, LPCSTR
szShaderModel, ID3DBlob** ppBlobOut)
{
    HRESULT hr = S_OK;
    ID3DBlob* pErrorBlob = NULL;

    hr = D3DCompileFromFile(szFileName, NULL, NULL, szEntryPoint, szShaderModel,
        D3DCOMPILER_ENABLE_STRICTNESS, 0, ppBlobOut, &pErrorBlob);

    if (FAILED(hr))
    {
        if (pErrorBlob)
        {
            OutputDebugStringA((char*)pErrorBlob->GetBufferPointer());
            pErrorBlob->Release();
        }
        return hr;
    }
    if (pErrorBlob) pErrorBlob->Release();

    return S_OK;
}

// Инициализация видеокарты и создание объектов
HRESULT InitDevice(HWND hwnd)
{
    HRESULT hr = S_OK;

    RECT rc;
    GetClientRect(hwnd, &rc);
    UINT width = rc.right - rc.left;
    UINT height = rc.bottom - rc.top;

    DXGI_SWAP_CHAIN_DESC sd = {};
    sd.BufferCount = 1;
    sd.BufferDesc.Width = width;
    sd.BufferDesc.Height = height;

```

```

sd.BufferDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
sd.BufferDesc.RefreshRate.Numerator = 60;
sd.BufferDesc.RefreshRate.Denominator = 1;
sd.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
sd.OutputWindow = hwnd;
sd.SampleDesc.Count = 1;
sd.SampleDesc.Quality = 0;
sd.Windowed = TRUE;

D3D_DRIVER_TYPE driverType = D3D_DRIVER_TYPE_HARDWARE;
D3D_FEATURE_LEVEL featureLevel = D3D_FEATURE_LEVEL_11_0;

hr = D3D11CreateDeviceAndSwapChain(NULL, driverType, NULL, 0, &featureLevel, 1,
    D3D11_SDK_VERSION, &sd, &g_pSwapChain, &g_pd3dDevice, NULL,
    &g_pImmediateContext);
if (FAILED(hr)) return hr;

ID3D11Texture2D* pBackBuffer = NULL;
hr = g_pSwapChain->GetBuffer(0, __uuidof(ID3D11Texture2D), (LPVOID*)&pBackBuffer);
if (FAILED(hr)) return hr;

hr = g_pd3dDevice->CreateRenderTargetView(pBackBuffer, NULL, &g_pRenderTargetView);
pBackBuffer->Release();
if (FAILED(hr)) return hr;

g_pImmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);

D3D11_VIEWPORT vp;
vp.Width = (FLOAT)width;
vp.Height = (FLOAT)height;
vp.MinDepth = 0.0f;
vp.MaxDepth = 1.0f;
vp.TopLeftX = 0;
vp.TopLeftY = 0;
g_pImmediateContext->RSSetViewports(1, &vp);

// Компиляция Вершинного Шейдера
ID3DBlob* pVSBlob = NULL;
hr = CompileShaderFromFile(L"shaders.hlsl", "VS", "vs_4_0", &pVSBlob);
if (FAILED(hr)) return hr;

hr = g_pd3dDevice->CreateVertexShader(pVSBlob->GetBufferPointer(), pVSBlob->
    GetBufferSize(), NULL, &g_pVertexShader);
if (FAILED(hr))
{
    pVSBlob->Release();
    return hr;
}

// Описываем формат вершин для видеокарты (Input Layout)
D3D11_INPUT_ELEMENT_DESC layout[] =
{
    { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0, D3D11_INPUT_PER_VERTEX_DATA,
0 },
    { "TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 12, D3D11_INPUT_PER_VERTEX_DATA, 0
},
};
UINT numElements = sizeof(layout) / sizeof(layout[0]);

hr = g_pd3dDevice->CreateInputLayout(layout, numElements, pVSBlob->
    GetBufferPointer(), pVSBlob->GetBufferSize(), &g_pVertexLayout);
pVSBlob->Release();
if (FAILED(hr)) return hr;

g_pImmediateContext->IASetInputLayout(g_pVertexLayout);

// Компиляция Пиксельного Шейдера
ID3DBlob* pPSBlob = NULL;
hr = CompileShaderFromFile(L"shaders.hlsl", "PS", "ps_4_0", &pPSBlob);

```

```

        if (FAILED(hr)) return hr;

        hr = g_pd3dDevice->CreatePixelShader(pPSBlob->GetBufferPointer(), pPSBlob-
>GetBufferSize(), NULL, &g_pPixelShader);
        pPSBlob->Release();
        if (FAILED(hr)) return hr;

        // Создаем тестовую текстуру 2x2 пикселя (Черный и Белый в шахматном порядке)
        UINT textureData[] = {
            0xFFFFFFFF, 0xFF000000,
            0xFF000000, 0xFFFFFFFF
        };

        D3D11_TEXTURE2D_DESC desc = {};
        desc.Width = 2;
        desc.Height = 2;
        desc.MipLevels = 1;
        desc.ArraySize = 1;
        desc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
        desc.SampleDesc.Count = 1;
        desc.Usage = D3D11_USAGE_DEFAULT;
        desc.BindFlags = D3D11_BIND_SHADER_RESOURCE;

        D3D11_SUBRESOURCE_DATA initData = {};
        initData.pSysMem = textureData;
        initData.SysMemPitch = 2 * sizeof(UINT);

        ID3D11Texture2D* pTexture = NULL;
        hr = g_pd3dDevice->CreateTexture2D(&desc, &initData, &pTexture);
        if (SUCCEEDED(hr))
        {
            hr = g_pd3dDevice->CreateShaderResourceView(pTexture, NULL, &g_pTextureSRV);
            pTexture->Release();
        }
        if (FAILED(hr)) return hr;

        // Настраиваем режим POINT-фильтрации (строго без размытия)
        D3D11_SAMPLER_DESC sampDesc = {};
        sampDesc.Filter = D3D11_FILTER_MIN_MAG_MIP_POINT;
        sampDesc.AddressU = D3D11_TEXTURE_ADDRESS_CLAMP;
        sampDesc.AddressV = D3D11_TEXTURE_ADDRESS_CLAMP;
        sampDesc.AddressW = D3D11_TEXTURE_ADDRESS_CLAMP;
        sampDesc.ComparisonFunc = D3D11_COMPARISON_NEVER;
        sampDesc.MinLOD = 0;
        sampDesc.MaxLOD = D3D11_FLOAT32_MAX;

        hr = g_pd3dDevice->CreateSamplerState(&sampDesc, &g_pSamplerState);
        if (FAILED(hr)) return hr;

        // Геометрия прямоугольного экрана
        SimpleVertex vertices[] =
        {
            { { -1.0f, 1.0f, 0.0f }, { 0.0f, 0.0f } },
            { { 1.0f, 1.0f, 0.0f }, { 1.0f, 0.0f } },
            { { -1.0f, -1.0f, 0.0f }, { 0.0f, 1.0f } },
            { { 1.0f, -1.0f, 0.0f }, { 1.0f, 1.0f } },
        };

        D3D11_BUFFER_DESC bd = {};
        bd.Usage = D3D11_USAGE_DEFAULT;
        bd.ByteWidth = sizeof(SimpleVertex) * 4;
        bd.BindFlags = D3D11_BIND_VERTEX_BUFFER;
        bd.CPUAccessFlags = 0;

        D3D11_SUBRESOURCE_DATA InitData = {};
        InitData.pSysMem = vertices;

        hr = g_pd3dDevice->CreateBuffer(&bd, &InitData, &g_pVertexBuffer);
        if (FAILED(hr)) return hr;

```

```

    return S_OK;
}

// Отрисовка кадра
void Render()
{
    if (g_pRenderTargetView == NULL) return;

    float ClearColor[] = { 0.75f, 0.75f, 0.75f, 1.0f };
    g_pImmediateContext->ClearRenderTargetView(g_pRenderTargetView, ClearColor);

    UINT stride = sizeof(SimpleVertex);
    UINT offset = 0;
    g_pImmediateContext->IASetVertexBuffers(0, 1, &g_pVertexBuffer, &stride, &offset);
    g_pImmediateContext->IASetPrimitiveTopology(D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP);

    g_pImmediateContext->VSSetShader(g_pVertexShader, NULL, 0);
    g_pImmediateContext->PSSetShader(g_pPixelShader, NULL, 0);

    g_pImmediateContext->PSSetShaderResources(0, 1, &g_pTextureSRV);
    g_pImmediateContext->PSSetSamplers(0, 1, &g_pSamplerState); // Связываем наш POINT-сэмплер

    g_pImmediateContext->Draw(4, 0);

    g_pSwapChain->Present(0, 0);
}

// Безопасное освобождение памяти
void CleanupDevice()
{
    if (g_pImmediateContext) g_pImmediateContext->ClearState();

    if (g_pSamplerState) { g_pSamplerState->Release(); g_pSamplerState = NULL; }
    if (g_pTextureSRV) { g_pTextureSRV->Release(); g_pTextureSRV = NULL; }
    if (g_pVertexBuffer) { g_pVertexBuffer->Release(); g_pVertexBuffer = NULL; }
    if (g_pVertexLayout) { g_pVertexLayout->Release(); g_pVertexLayout = NULL; }
    if (g_pPixelShader) { g_pPixelShader->Release(); g_pPixelShader = NULL; }
    if (g_pVertexShader) { g_pVertexShader->Release(); g_pVertexShader = NULL; }
    if (g_pRenderTargetView) { g_pRenderTargetView->Release(); g_pRenderTargetView = NULL; }
    if (g_pSwapChain) { g_pSwapChain->Release(); g_pSwapChain = NULL; }
    if (g_pImmediateContext) { g_pImmediateContext->Release(); g_pImmediateContext = NULL; }
    if (g_pd3dDevice) { g_pd3dDevice->Release(); g_pd3dDevice = NULL; }
}

// Главная точка входа Windows-приложения
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg, WPARAM wParam, LPARAM lParam)
{
    switch (uMsg)
    {
    case WM_SIZE:
        if (g_pSwapChain)
        {
            if (g_pRenderTargetView) { g_pRenderTargetView->Release(); }
            g_pRenderTargetView = NULL;
            UINT width = LOWORD(lParam);
            UINT height = HIWORD(lParam);
            g_pSwapChain->ResizeBuffers(0, width, height, DXGI_FORMAT_UNKNOWN, 0);

            ID3D11Texture2D* pBackBuffer = NULL;
            g_pSwapChain->GetBuffer(0, __uuidof(ID3D11Texture2D),
            (LPVOID*)&pBackBuffer);
            if (pBackBuffer)
            {
                g_pd3dDevice->CreateRenderTargetView(pBackBuffer, NULL,
                &g_pRenderTargetView);
            }
        }
    }
}

```

```

        pBackBuffer->Release();
    }
    g_pImmediateContext->OMSetRenderTargets(1, &g_pRenderTargetView, NULL);

    D3D11_VIEWPORT vp;
    vp.Width = (FLOAT)width;
    vp.Height = (FLOAT)height;
    vp.MinDepth = 0.0f;
    vp.MaxDepth = 1.0f;
    vp.TopLeftX = 0;
    vp.TopLeftY = 0;
    g_pImmediateContext->RSSetViewports(1, &vp);
}
return 0;

case WM_DESTROY:
    PostQuitMessage(0);
    return 0;
}
return DefWindowProc(hwnd, uMsg, wParam, lParam);
}

```